

MAD 2 Project Report

Vehicle Parking App - V2

1. Student Details

Name: Ikram Abdulrehman Khan

Roll Number: 23F2002017

Email: 23f2002017@ds.study.iitm.ac.in

About Me: I am a full time student currently pursuing this course of Data Science and Applications from IIT Madras as my standalone Degree. I've always had a really deep interest in Applied Mathematics and application of these ideas in the field of Computers Science and Machine Learning. Throughout my journey in this course I've also found myself enjoying the process of building these meaningful applications that combine learning, analytics, and user experience.

2. Project Details

Project Title: Vehicle Parking Application V2

Problem Statement:

To design and build a multi-user web-based application that allows users to **locate and book parking spots easily on-the-fly**. The **administrator (or the superuser)** of the application must be able **track parkings, manage parking lots and view all the users that are currently logged on to the application**.

Approach:

The app is built using Flask as the backend framework with a modular structure. The UI of the application is built using VueJS as the frontend framework. Caching has been implemented in the application using Redis (an in-memory database). Celery is used to execute scheduled and backend jobs for the application with Redis serving as both the Task Queue and Results Backend.

3. AI/LLM Declaration

The use of AI in the project has been limited only to **code suggestions by VSCode** which might be about **5-10%** of the entire code. The code for configuration of certain libraries like Celery, Flask-Caching, etc has been referenced (and sometimes copied) from the sites containing official documentation of these libraries.

There is no use of AI/LLM for code generation in the entire project.

4. Technologies and Frameworks Used

Technology/Library	Purpose
Flask	Core backend web framework
SQLite	Lightweight local database for storing user data
Flask-SQLAlchemy	Flask Extension for SQLAlchemy, an Object Relational Mapper for SQLite database
Flask-Security-Too	For implementing authentication and RBAC
VueJS	A frontend JavaScript framework for UI
Celery	For task scheduling and handling backend jobs
Flask-Caching	For implementation of caching in the app
Redis	An in-memory database working as a Task Queue, Results backend and cache storage
Matplotlib	To create charts for displaying summary on stats
Jinja2	Templating engine for creating the monthly activity report
smtplib	To create an SMTP for sending mails
datetime & pytz	To get the current timestamps for our timezone

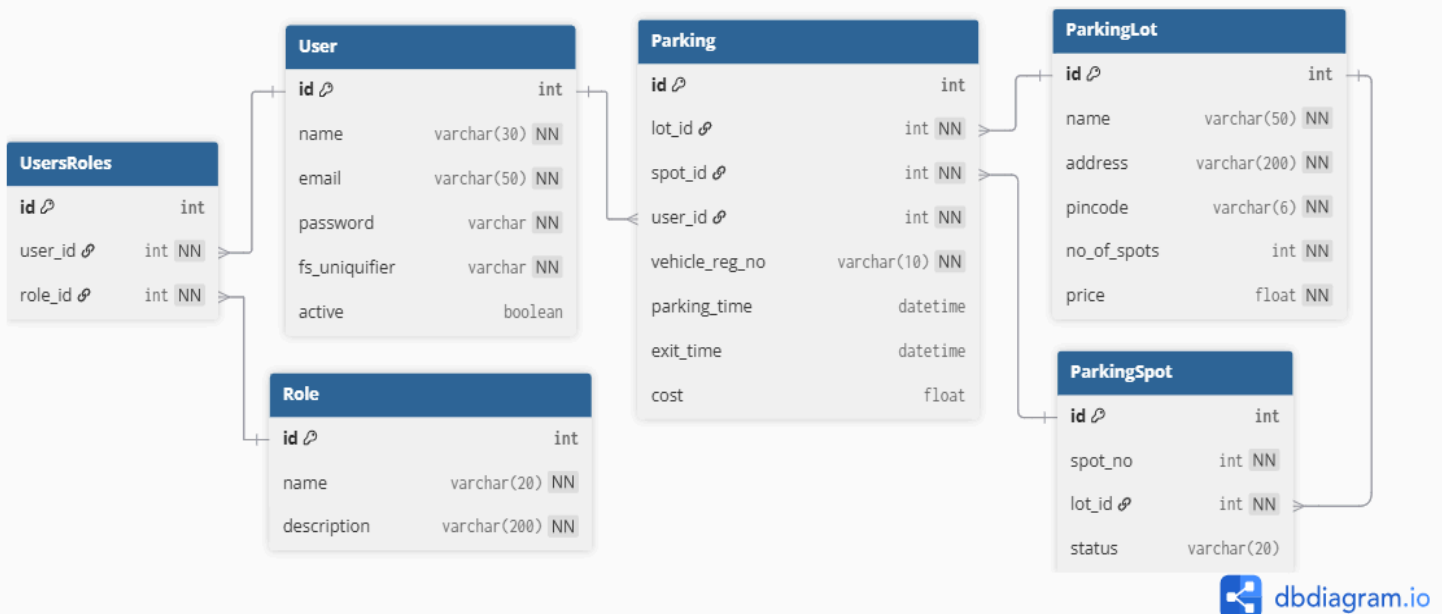
5. Database Schema / ER Diagram

Tables:

1. **User** — stores user profile details (id, name, email, password, status)
2. **Role** — stores different roles (id, name, description)
3. **UsersRoles** — contains mapping of users with roles (id, user_id, role_id)
4. **ParkingLot** — stores info on parking lot (id, name, address, pincode, no_of_spots, price)
5. **ParkingSpot** — stores status of every parking spot (id, spot_no, lot_id, status)
6. **Parking** — stores information about every parking (id, lot_id, spot_id, user_id, vehicle_reg_no, parking_time, exit_time, cost)

Relationships:

- Many-to-Many → **User** → **Role**
- One-to-Many → **User** → **Parking**
- One-to-Many → **ParkingLot** → **ParkingSpot**
- One-to-Many → **ParkingLot** → **Parking**
- One-to-Many → **ParkingSpot** → **Parking**



6. API Resource Endpoints

Endpoint	Method	Description
<code>/api/login</code>	POST	To authenticate user
<code>/api/logout</code>	GET	To logout the current user
<code>/api/register</code>	POST	To register a new user
<code>/api/admin</code>	GET	To get the list of all parking lots
<code>/api/users</code>	GET	To get the list of all the users
<code>/api/users/change_status/<user_id></code>	PUT	To block/unblock a user
<code>/api/parkings</code>	GET	To get the list of all the parkings
<code>/api/parking_lot</code>	POST	To add a new parking lot
<code>/api/parking_lot/<lot_id></code>	GET	To get the details of a parking lot
<code>/api/parking_lot/<lot_id></code>	PUT	To edit a parking lot
<code>/api/parking_lot/<lot_id></code>	DELETE	To delete a parking lot
<code>/api/parking_spot/<spot_id></code>	GET	To view a parking spot
<code>/api/parking_spot/<spot_id></code>	DELETE	To delete a parking spot
<code>/api/search</code>	POST	To add search for admin/user
<code>/api/summary</code>	GET	To get some sort of summary
<code>/api/status</code>	GET	To get the status of a user
<code>/api/profile</code>	GET	To get details of a user
<code>/api/profile</code>	PUT	To edit a user profile
<code>/api/user</code>	GET	List of parking lots and current parkings
<code>/api/parking_history</code>	GET	To get a list of all the past parkings
<code>/api/book_parking<lot_id></code>	POST	To book a parking spot
<code>/api/release_parking/<parking_id></code>	PUT	To release the parking spot

8. Video Presentation

Drive Link: [MAD 2 Project Video](#)